

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Patteela Yashwanth Kumar	Reg.No.:	192471055
Department:	CSE-BIOSCIENCE	Date:	29/08/26

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

*I Patteela Yashwanth Kumar/192471055 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Patteela Yashwanth Kumar

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. DMA-Based Data Transfer

Aim

To design a DMA-based algorithm that transfers data from an I/O device to memory with interrupt handling and minimum CPU involvement.

Algorithm

1. Initialize DMA controller.
2. Set source as I/O device.
3. Set destination as memory.
4. Set the transfer size.
5. Start DMA transfer.
6. CPU performs other tasks while DMA transfers data.
7. DMA generates an interrupt after completion.
8. CPU services the interrupt and checks the transfer status.
9. Report completion.

Pseudocode

BEGIN

 Initialize DMA_Controller
 Set Source = IO_Device
 Set Destination = Memory
 Set Transfer_Size = N

 Start_DMA_Transfer()

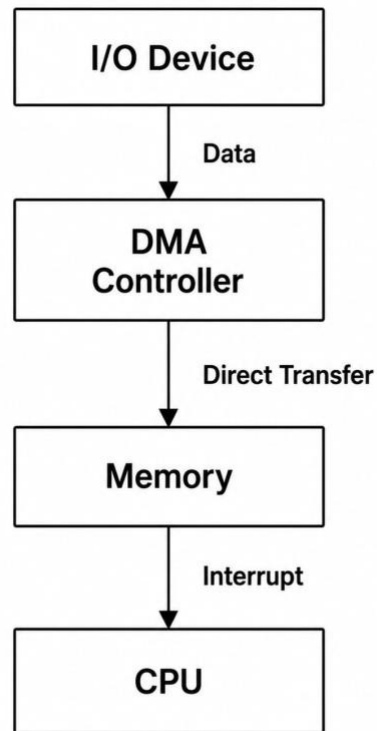
 WHILE DMA_Transfer_Not_Complete
 CPU_Perform_Other_Tasks()
 END WHILE

 ON DMA_Interrupt:
 Check_Transfer_Status()

 IF Status = SUCCESS THEN
 Display "DMA Transfer Completed"
 ELSE
 Display "DMA Transfer Failed"
 END IF

END

Diagram



Sample Output

DMA Transfer Started
CPU Performing Other Tasks
DMA Interrupt Received
DMA Transfer Completed Successfully

Result

The DMA algorithm transfers data directly between the I/O device and memory while reducing CPU involvement.

2. Vectored Interrupt Handling

Aim

To design an interrupt handling system that prioritizes vectored interrupts from multiple devices and executes the appropriate service routine.

Algorithm

1. Initialize interrupt controller.
2. Receive interrupt requests from devices.
3. Check the priority of each interrupt.
4. Select the highest-priority interrupt.
5. Obtain its interrupt vector.
6. Execute the corresponding ISR.
7. Clear the interrupt.
8. Check for remaining interrupts.

Pseudocode

BEGIN

Initialize Interrupt_Controller

WHILE System_Running

IF Interrupt_Received THEN

Identify all Pending_Interrupts

Select Highest_Priority_Interrupt

Vector = Get_Interrupt_Vector()

Execute ISR(Vector)

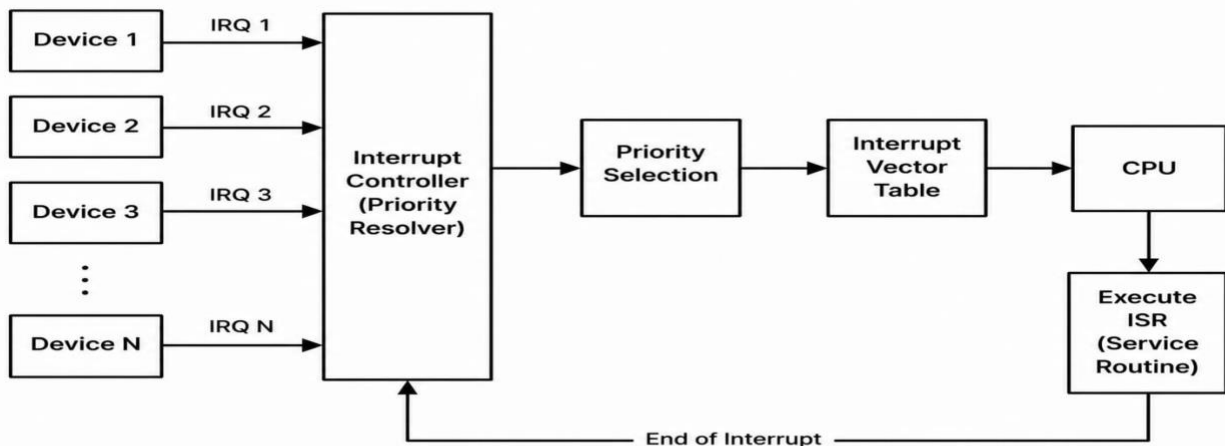
Clear Interrupt

END IF

END WHILE

END

Diagram



Sample Input

Interrupts Received: Device 1, Device 2, Device 3

Highest Priority: Device 2

Vector Identified: 02

Executing ISR for Device 2

Interrupt Served Successfully

Result

The system successfully prioritizes multiple vectored interrupts and executes the correct service routine.

3. Dynamic Selection of Memory-Mapped and I/O-Mapped I/O

Aim

To dynamically select between memory-mapped I/O and I/O-mapped I/O according to latency and bandwidth requirements.

Algorithm

1. Read latency and bandwidth requirements.
2. Compare them with predefined thresholds.
3. Select memory-mapped I/O for high bandwidth/low latency requirements.
4. Otherwise select I/O-mapped I/O.
5. Perform the required I/O operation.

Pseudocode

BEGIN

 Read Latency

 Read Bandwidth

 IF Latency $\leq$ LOW_LATENCY AND Bandwidth $\geq$ HIGH_BANDWIDTH THEN

 Mode = MEMORY_MAPPED_IO

 ELSE

 Mode = IO_MAPPED_IO

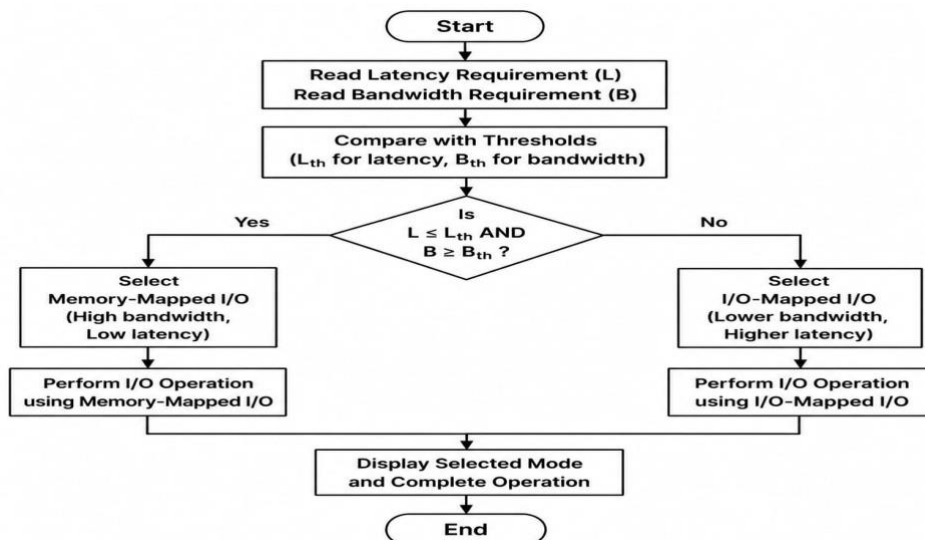
 END IF

 Perform_IO_Operation(Mode)

 Display Selected_Mode

END

Diagram



Sample Output

Latency = Low

Bandwidth = High

Selected Mode = Memory-Mapped I/O

I/O Operation Completed

Result

The algorithm dynamically selects a suitable I/O technique based on latency and bandwidth requirements.

4. Bus Arbitration for PCIe, USB and Thunderbolt

Aim

To design a bus arbitration algorithm that manages multiple devices communicating through PCIe, USB, and Thunderbolt interfaces.

Algorithm

1. Initialize the bus controller.
2. Receive requests from connected devices.
3. Identify the interface and priority.
4. Select the highest-priority request.
5. Grant bus access.
6. Allow the device to transfer data.
7. Release the bus after completion.
8. Repeat for pending requests.

Pseudocode

BEGIN

 Initialize Bus_Controller

 WHILE System_Running

 Collect Requests from PCIe, USB, Thunderbolt

 IF Requests_Available THEN

 Select Highest_Priority_Request

 Grant_Bus_Access()

 Perform_Data_Transfer()

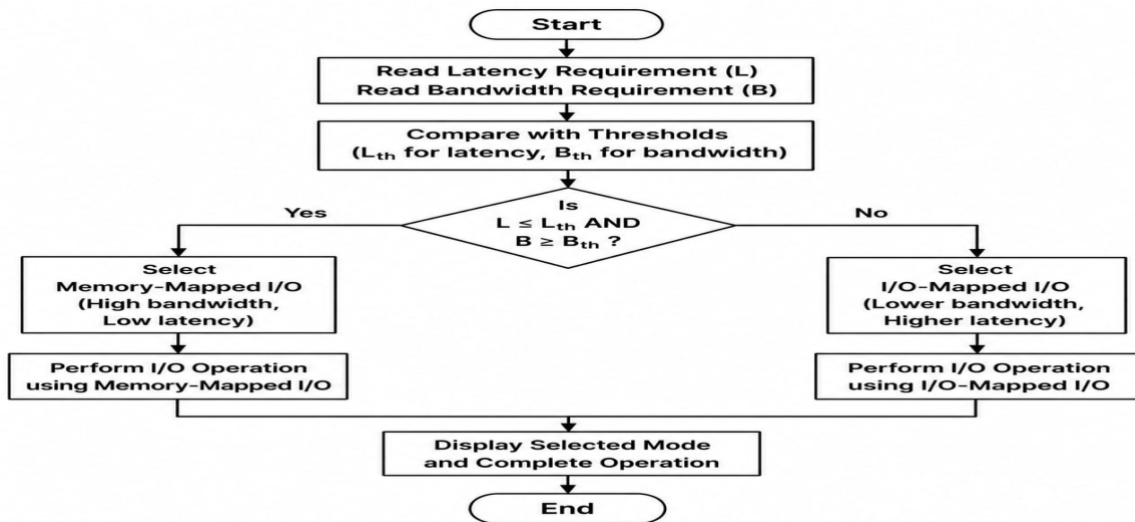
 Release_Bus()

 END IF

 END WHILE

END

Diagram



Sample Output

Requests: PCIe, USB, Thunderbolt

Selected Interface: PCIe

Bus Access Granted

Data Transfer Completed

Bus Released

Next Request: USB

Result

The bus arbitration algorithm manages competing device requests and provides controlled bus access to PCIe, USB, and Thunderbolt devices.

5. Hybrid I/O Communication Mechanism

Aim

To implement a hybrid I/O mechanism using polling for low-speed devices and DMA with interrupts for high-speed devices.

Algorithm

1. Identify the device speed.
2. If the device is low-speed, use polling.
3. If the device is high-speed, configure DMA.
4. Start DMA transfer.
5. CPU continues other operations.
6. DMA generates an interrupt after completion.
7. Service the interrupt and report the result.

Pseudocode

BEGIN

 Identify Device

 IF Device_Speed = LOW THEN

 WHILE Data_Not_Ready

 Poll_Device()

 END WHILE

 Read_Data()

 Display "Polling Transfer Completed"

 ELSE

 Initialize DMA

 Configure DMA

 Start_DMA_Transfer()

 CPU_Perform_Other_Tasks()

 ON DMA_Interrupt:

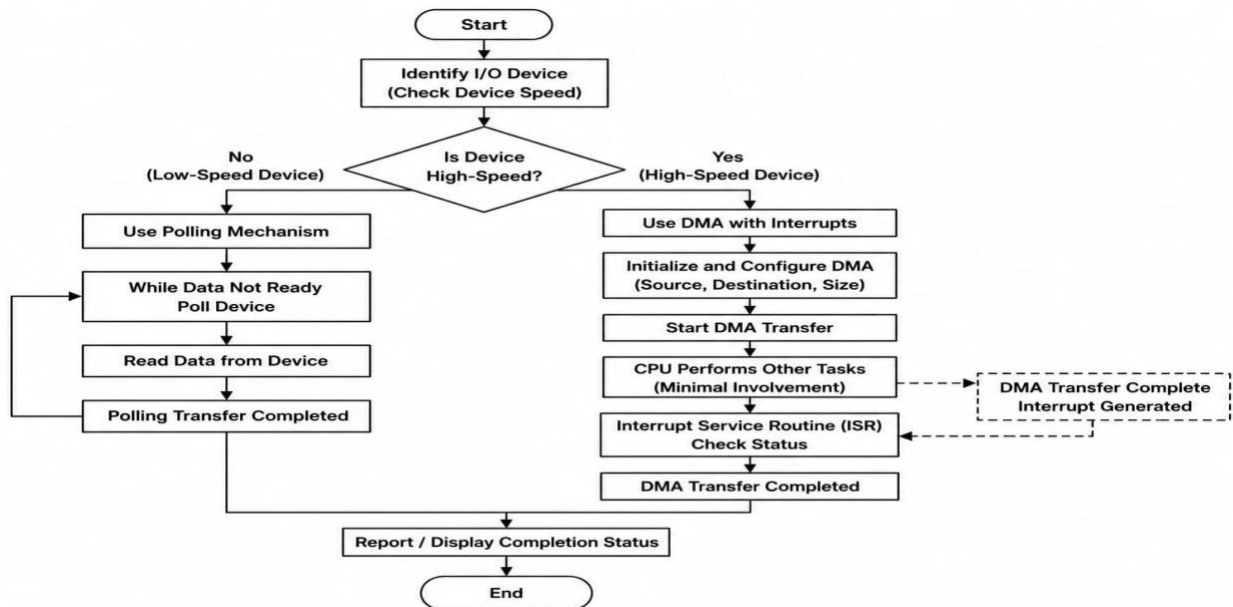
 Check_Status()

 Display "DMA Transfer Completed"

 END IF

END

Diagram



Sample Output

Device Speed: Low

Mode Selected: Polling

Polling Transfer Completed

For a high-speed device:

Device Speed: High

Mode Selected: DMA

DMA Transfer Started

DMA Interrupt Received

DMA Transfer Completed

Result

The hybrid mechanism efficiently handles both low-speed and high-speed devices by using polling for low-speed communication and DMA with interrupts for high-speed communication.